Computer Programming Contest

Programming Competition

Contestant Packet

April-2026

Programming Guidelines

- All programs must include internal documentation so that they can be supported by others after they are deployed. All classes and methods should fully document the inputs and outputs.
- All programs input should be checked for security and should provide adequate errors in the event of invalid input.
- While solutions can include a compiled EXE, the executable should be re-creatable from the committed code.
- The programs should be self-contained such that everything required to compile and run the program is included in the repository, as well as any documentation required to perform the compilation.
- **For the final challenge a small database must be used, it can either be a TXT file export/import, or a sqllite3 (bonus points)**
- You may use standard Toolkits, API's, Frameworks. So long as they are used in supporting the creation of the application and are not the application themselves. Things included in the Standard Packages are allowed, as Java SE includes JDBC, JFC, Junit, JavaDoc. Other **packages like Thymeleaf, Typescript may be used as well. Math libraries are also needed but may not be used to solve the final equation.**
- If you are including external libraries, be sure to package the solution with an Ant, Maven, package.json, requirements.txt or similar build system to simplify deployment.
- You shall create a separate directory in your Git repo for each project, the code related to each project be clear.
- Depending on the programming language, please include a README.txt describing how to compile it. For example for python include a requirements.txt file with all packages needed to load from pip. For Javascript please include a package.json with all packages needed.
- If your program will run in Docker, please include the Dockerfile with the instructions to build it in the README.txt file.
- **Make sure to pay attention to the scoring rubric to get as many points as possible.**
- Most scoring will be completed when you have uploaded your Evidence to the Git Repo.
- If you are unable to upload to a git-repo, please talk to the Tech Chair for alternative options.
- All code, and evidence must be **submitted by 3pm on the** day of the competition. After you submit your code, let the tech chair know so we can verify we can see and judge it. Missing or incomplete evidence of program execution and testing will result in a 0.
- **For Full points, challenges 1 and 2 must be uploaded by noon. You must complete 1 and 2, before you can submit 3.**
- **If you are coding in an "Online Tool" the Tech Chair will inspect the solution on your computer.**

Scoring Rubric

Title	Points	Percentage	Total for Section
Program 1: CLI : Completeness	50	20%	
Program 1: CLI : Correctness of Output	50		
Program 1: CLI : Validation of Input	40		
Program 1: CLI : Code Comments	40		
Program 1: CLI : Quality of Work	20		200
Program 2: API : Completeness	75	20%	
Program 2: API : Correctness of Output	55		
Program 2: API : Validation of Input	25		
Program 2: API : Code Comments	25		
Program 2: API : Quality of Work	20		200
Program 3: Cash : Completeness	75	45%	
Program 3: Cash : Client/Server/BFF API	75		
Program 3: Cash : Validation of Input	75		
Program 3: Cash : Reports Quality	75		
Program 3: Cash : Database / Storage	75		
Program 3: Cash : Usability / UI	75		450
Late Submission (0 or -100)			
Online Test	150	15%	150
Résumé Penalty (0 or -50)			
Clothing Penalty (0 to -20)			
Total Possible Points	1,000		

Program #1 – CLI Program

Project: Fibonacci sequence generator (CLI Program)

Project Goal: You will write a command line program to generate Fibonacci sequences on demand. The program will take several arguments, defined below, and will output the corresponding numbers. Definition: According to mathisfun.com: The **Fibonacci Sequence** is the series of numbers: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ... The next number is found by adding up the two numbers before it.

Requirements: You will run the program from a command prompt, and pass in arguments as defined in this help output:

./Fibonacci-gen [-c|--count] 20 –one-line –last-only –numbering –help

Help for Fibonacci generator:

--help : Print this help

--count|-c : Calculate to this many places. IE: 0, 1, 1, 2, 3, 5 would be the result of -c 6

--one-line: Print all the numbers on one line, separated by commas. Without this option, each number in the sequence will be printed on a new line.

--numbering: Preface each number in the sequence with it's placement: IE for "-c 6 –numbering –one-line" you would get this: "1:0, 2:1, 3:1, 4:2, 5:3, 6:5" where the first number is the count and the second is the Fibonacci sequence. Note: this argument should work with all other arguments.

--last-only: Only print the last number in the sequence

Program #1 – CLI Program – page 2

TESTING:

Please run the following tests and record the output.

A) **.**/Fibonacci-gen -c 6 –one-line –numbering
    a. **1:0, 2:1, 3:1, 4:2, 5:3, 6:5**
B) **.**/Fibonacci-gen –help
    a. **Should output the above help**
C) **.**/Fibonacci-gen -c 6 –one-line
    a. **0, 1, 1, 2, 3, 5**
D) **.**/Fibonacci-gen --count 6
    a. **0**
    b. **1**
    c. **1**
    d. **2**
    e. **3**
    f. **5**
E) **.**/Fibonacci-gen --count 15 –last-only
    a. 377
F) **.**/Fibonacci-gen --count 50 –last-only
    a. (figure it out)

When you have completed this program, please commit all of your code to GitHub, as well as a PowerPoint or similar evidence that clearly shows your program running with proper output. Clearly show in the presentation the testing for all the test cases. List your name and Program Name on your Power Point or evidence and commit that into your Git Repo.

Program #2 – API Client

Project: Zip Code lookup and Weather

Project Goal: You will write a client program that performs API calls to the internet to get the weather forecast. You will need to read the documentation on api.weather.gov, but in short you will need to convert a zip-code to a latitude and longitude.

Requirements:

- You will use https://zippopotam.us/ for zip code lookups to latitude and longitude
- Then use the points API at https://api.weather.gov/ to get the forecast information
- Then you will retrieve the forecast and display it.
- The program will accept a zip code, and will then print the City, State, Lat, Log.
- It will further fetch the points api, like this:
  https://api.weather.gov/points/42.345235,-88.029861
- And then process it to get the forecast, and forecastHourly and display that. In a Pretty HTML output, Images for full points.

You will need to read the OpenAPI Specification to understand how to program against it. You MAY use Bruno or Postman to assist with inspecting the API. You may not use those tools code generation functions.

When you have completed this program, please commit all of your code to GitHub, as well as a PowerPoint or similar evidence that clearly shows your program running with proper output. Clearly show in the presentation the testing for all the test cases. List your name and Program Name on your Power Point or evidence and commit that into your Git Repo.

Program #3

Project: Cash Register Program.

Project Goal: You will write a program to act as a cash register for a small business. It will provide two modes of operation. Admin: to allow the owner to add items to inventory with a SKU, price and number on hand. Cashier: To scan items and get their price, subtract from inventory once the "Paid" button is clicked. It should calculate the sales tax on the sale of 5.5%, and record in a table the amount due to the State.

Requirements:

- Be able to add items to inventory. Add at least 10 items to inventory. Each item should have a. Description, Unit Cost, Sale Price, number on hand.
- In Admin mode you will be able to print an output to the screen of the: Current Inventory of items. Units on hand, Cost of the Item (COGS), Sale Price, Projected Profit.
- You should also be able to run a report on the days Sales: Which should report on the same type of data from above.
- In cashier mode. When you enter a SKU, it will ask for a quantity to sell. And it will show you the total for the sale. The inventory deduction should not happen until the order is "Completed". So there should be a "Cancel" order button which will return to allow another checkout to happen.

Your customer has provided minimal requirements as they don't know what they want, you will be graded on the completeness of your solution, considering the lack of requirements.

When you have completed this program, **please commit all of your code to GitHub**, as well as a PowerPoint or similar evidence that clearly shows your program running with proper output. **Raise your hand and the instructor will come over and review your code running on your laptop.** List your name and Program Name on your Power Point or evidence and commit that into your Git Repo.